



Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

SWE 4802: Software Maintenance Lab

Summer, 2024-2025

Lab 3

Transformation Tools

Namisa Najah Raisa

210042112

Program: B.Sc in SWE

Department of CSE

Date: June 14, 2026

Contents

1	Usage of the tools in the project	2
1.1	Loguru	2
1.1.1	Usage in project	2
1.1.2	Insight from output	3
1.1.3	Maintenance use case	3
1.2	Pysnooper	3
1.2.1	Usage in project	3
1.2.2	Insight from output	4
1.2.3	Maintenance use case	4
1.3	CProfile	4
1.3.1	Usage in project	4
1.3.2	Insight from output	5
1.3.3	Maintenance use case	5
1.4	Snakeviz	5
1.4.1	Usage in project	5
1.4.2	Insight from output	6
1.4.3	Maintenance use case	6
1.5	Viztracer	6
1.5.1	Usage in project	6
1.5.2	Insight from output	7
1.5.3	Maintenance use case	7

1 Usage of the tools in the project

1.1 Loguru

1.1.1 Usage in project

integrated Loguru into my project to replace print() statements used for debugging. I used logger.info(), logger.debug(), and logger.error() at key points (e.g., function entry, API calls, error handling) to track program flow and capture runtime errors with stack traces using logger.exception().

1. Loguru terminal output

```
PS C:\Users\Dell\Desktop\maintenance> python main.py
X Please enter a whole number.

10:55:39 | WARNING | __main__:main - Invalid menu input: ''

Welcome To Student Management System

1. View all students
2. Add a new student
3. Search for a student
4. Remove a student
5. Update a student
6. Full report (stats + classification)
7. Sort students
0. Exit

Select option: 1
10:55:43 | DEBUG | __main__:main - Menu choice: 1
10:55:43 | INFO | __main__:action_view - User selected: View all students.
10:55:43 | INFO | services.student_service:list_students - Listing students (enrolled_only=False).
10:55:43 | DEBUG | services.student_service:load_data - Loaded 6 student(s) from C:\Users\Dell\Desktop\maintenance\data\students.json.
10:55:43 | DEBUG | services.student_service:list_students - Returning 6 student(s).

— All Students (6) —

[STU001] Yugesh Kumar | Major: Computer Science | GPA: 3.80 | Status: Active | Standing: Distinction
[STU002] Kishor Patel | Major: Electrical Engineering | GPA: 3.20 | Status: Active | Standing: Merit
[STU003] Gajen Silva | Major: Mathematics | GPA: 2.50 | Status: Active | Standing: Pass
[STU004] Gopi Nair | Major: Physics | GPA: 1.80 | Status: Active | Standing: At Risk
[STU005] Amara Hassan | Major: Computer Science | GPA: 3.95 | Status: Active | Standing: Distinction
[STU0002] Namisa | Major: Swe | GPA: 3.80 | Status: Active | Standing: Distinction

Press Enter to continue...
```

2. Logger debug

```
logger.debug(f"Menu choice: {choice}")
ACTIONS[choice]()
```

3. Logger Info

```
@logger.catch(level="ERROR", message="Error in view_students")
def action_view():
    logger.info("User selected: view all students.")
    students = list_students()
    print(f"\n — All Students ({len(students)}) —")
    _print_students(students)
```

4. Logger success

```
print("\n" + "=" * 56 + "\n")
logger.success("Full report printed.")
```

5. Logger Warning

```
except ValueError as e:
    logger.warning(f"Add failed: {e}")
    print(f"\n X Error: {e}\n")
```

6. Logger Error

```
except (json.JSONDecodeError, KeyError) as exc:
    logger.error(f"Failed to parse data file: {exc}")
    return []
```

7. Logger Catch

```
@logger.catch(level="ERROR", message="Error in view_students")
def action_view():
    logger.info("User selected: view all students.")
    students = list_students()
    print(f"\n — All Students ({len(students)}) —")
    _print_students(students)
```

8. Logger file (student_system.log)

```
2026-06-11 18:32:53 | INFO | _main_:action add:87 | User selected: Add new student.
2026-06-11 18:33:03 | INFO | services.student_service:add_student:73 | Attempting to add student: name='namisa', id='1243'.
2026-06-11 18:33:03 | WARNING | utils.validators:validate_student_id:38 | Invalid student ID format: '1243'
2026-06-11 18:33:03 | WARNING | _main_:action add:97 | Add failed: Student ID must start with 1-4 letters followed by 2-8 digits (e.g. STU001).
2026-06-11 18:33:09 | DEBUG | _main_:main:192 | Menu choice: 2
2026-06-11 18:33:09 | INFO | _main_:action add:87 | User selected: Add new student.
2026-06-11 18:33:17 | INFO | services.student_service:add_student:73 | Attempting to add student: name='namisa', id='stu0001'.
2026-06-11 18:33:17 | DEBUG | services.student_service:load_data:29 | Data file not found at C:\Users\Dell\Desktop\maintenance\data\students.json
2026-06-11 18:33:17 | DEBUG | services.student_service:save_data:48 | Saved 1 student(s) to C:\Users\Dell\Desktop\maintenance\data\students.json
2026-06-11 18:33:17 | SUCCESS | services.student_service:add_student:93 | Added: [STU0001] Namisa | Major: Swe | GPA: 3.80 | Status: Active | Stand
```

1.1.2 Insight from output

The output shows timestamped, color-coded log entries with severity levels (**INFO**, **DEBUG**, **ERROR**), module names, and line numbers. This made it easy to trace exactly where and when an issue occurred, and to understand the sequence of operations without manually inserting print statements.

1.1.3 Maintenance use case

Loguru can be configured to write logs to a rotating file (**logger.add("file.log", rotation="1 week")**). During maintenance, these persistent logs help developers diagnose production issues after the fact, checking error logs to identify recurring exceptions or performance bottlenecks without needing to reproduce the bug live.

1.2 Pysnooper

1.2.1 Usage in project

I applied the **@pysnooper.snoop()** decorator to a function in my project that had unexpected output. This allowed me to trace every line executed inside the function along with the values of variables as they changed, without modifying the function's internal logic.

```
@pysnopper.snoop("pysnopper_trace.log")
def list_students(enrolled_only: bool = False) -> List[Student]:
    """
    Return all students (or only enrolled ones).
    PySnooper traces the filtering loop.
    """
    logger.info(f"Listing students (enrolled_only={enrolled_only}).")
    students = _load_data()
    if enrolled_only:
        students = [s for s in students if s.enrolled]
    logger.debug(f"Returning {len(students)} student(s).")
    return students
```

1.2.2 Insight from output

The output displayed each executed line in order, showing variable assignments and their updated values at every step. This gave a clear, step-by-step view of how data transformed through the function, making it easy to pinpoint exactly where a variable received an incorrect value or where the logic deviated from expectations.

Pysnooper output file (pysnopper_tracer.log):

```
New var:..... students = [Student(name='Yugesh Kumar', student_id='STU001...d=True, created_at='2026-06-11T18:39:52.606491')]
11:08:22.929000 line    61   if enrolled_only:
11:08:22.930001 line    63   logger.debug(f"Returning {len(students)} student(s).")
11:08:22.931006 line    64   return students
11:08:22.931999 return   64   return students
Return value:.. [Student(name='Yugesh Kumar', student_id='STU001...d=True, created_at='2026-06-11T18:39:52.606491')]
Elapsed time: 00:00:00.006989
```

1.2.3 Maintenance use case

When maintaining legacy code with unclear logic, Pysnooper can be temporarily attached to confusing or error-prone functions to quickly understand their behavior and variable states without spending time setting up a full debugger session—useful for quick diagnosis of reported bugs.

1.3 CProfile

1.3.1 Usage in project

I ran cProfile on a script in my project that was performing slowly, using cProfile via command line (**python -m cProfile models/student.py**) to generate a profile of all function calls during execution.

```

PS C:\Users\Bell\Desktop\maintenance> python -m cProfile models/student.py
1387 function calls (1373 primitive calls) in 0.003 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
5/1      0.000    0.000    0.003    0.003 {built-in method builtins.exec}
1        0.000    0.000    0.003    0.003 student.py:1(<module>)
3/1      0.000    0.000    0.002    0.002 <frozen importlib._bootstrap>:1165(_find_and_load)
3/1      0.000    0.000    0.002    0.002 <frozen importlib._bootstrap>:1120(_find_and_load_unlocked)
3/1      0.000    0.000    0.002    0.002 <frozen importlib._bootstrap>:666(_load_unlocked)
1        0.000    0.000    0.002    0.002 <frozen importlib._bootstrap_external>:934(exec_module)
1        0.000    0.000    0.001    0.001 <frozen importlib._bootstrap_external>:1007(get_code)
6/2      0.000    0.000    0.001    0.000 <frozen importlib._bootstrap>:233(_call_with_frames_removed)
1        0.000    0.000    0.001    0.001 datetime.py:1(<module>)
3        0.000    0.000    0.001    0.000 <frozen importlib._bootstrap>:1054(_find_spec)
1        0.000    0.000    0.001    0.001 <frozen importlib._bootstrap_external>:1499(find_spec)
1        0.000    0.000    0.001    0.001 <frozen importlib._bootstrap_external>:1467(_get_spec)
1        0.000    0.000    0.001    0.001 dataclasses.py:1204(dataclass)
1        0.000    0.000    0.001    0.001 dataclasses.py:1221(wrap)
1        0.000    0.000    0.001    0.001 dataclasses.py:884(_process_class)
1        0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1127(get_data)
4        0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1607(find_spec)
1        0.000    0.000    0.000    0.000 {built-in method io.open_code}
3        0.000    0.000    0.000    0.000 dataclasses.py:413(_create_fn)
1        0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:727(_compile_bytecode)
1        0.000    0.000    0.000    0.000 {built-in method marshal.loads}

```

1.3.2 Insight from output

The output listed each function along with the number of calls (ncalls), total time spent (tottime), cumulative time including sub-calls, and time per call. This helped identify which specific functions consumed the most execution time, revealing that certain functions were called far more often than expected or took disproportionately long, pointing directly to performance bottlenecks.

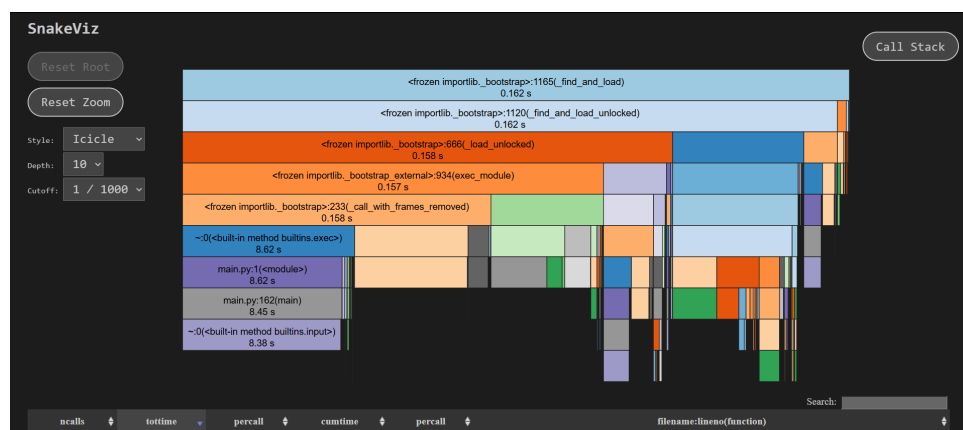
1.3.3 Maintenance use case

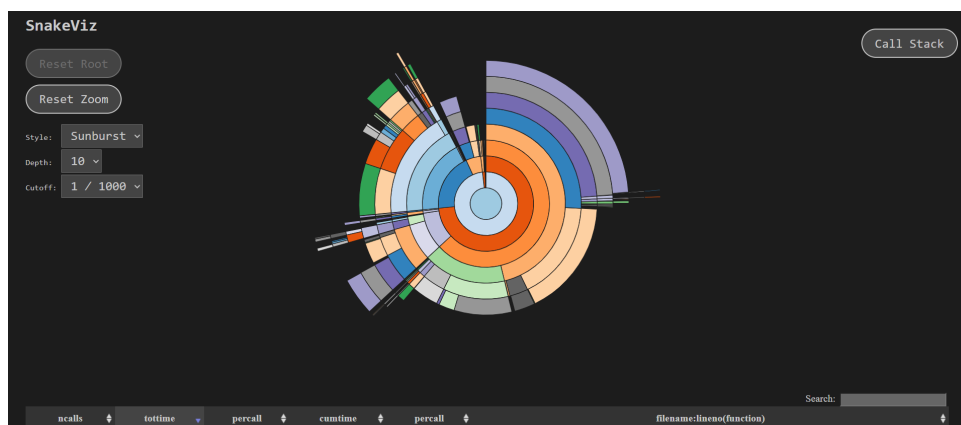
During maintenance, when a system update or new feature causes performance regressions, cProfile can be re-run to compare function-level timings before and after changes, quickly isolating which new or modified function is responsible for the slowdown.

1.4 Snakeviz

1.4.1 Usage in project

After profiling my script with cProfile and saving the output to a dotprofile file, I ran **snakeviz result.profile**, which opened a browser-based interactive visualization showing the call structure of my program.





1.4.2 Insight from output

The visualization displayed each function as a segment sized proportionally to its execution time, nested according to the call hierarchy. This made it immediately visible which functions and their sub-calls consumed the largest portions of total runtime, allowing me to quickly drill down into the most time-expensive call paths rather than scanning through raw text output.

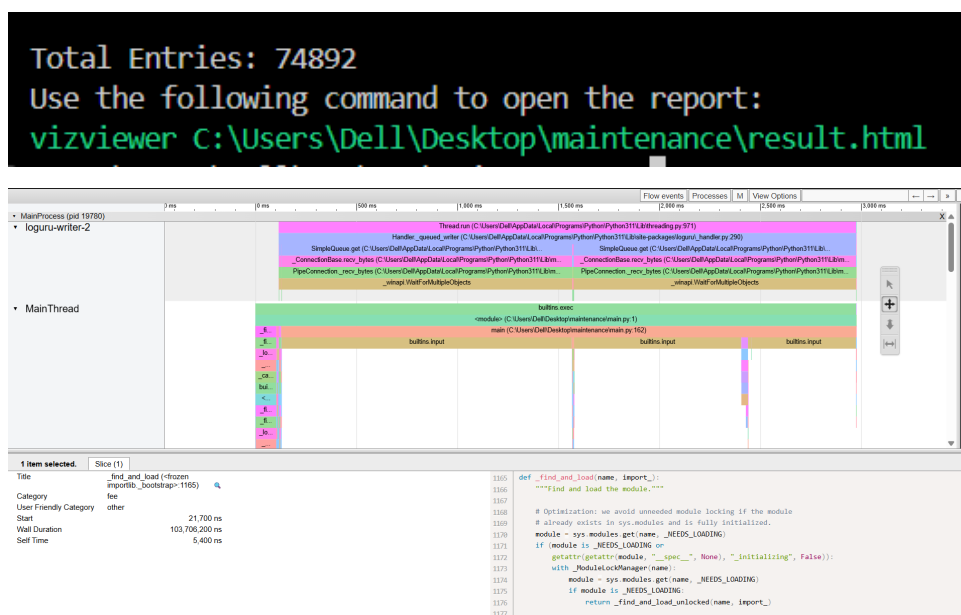
1.4.3 Maintenance use case

When performance issues are reported after deployment, Snakeviz can be used to visually compare profiling results across versions, quickly spotting which function's time share has grown and needs optimization—much faster than reading raw cProfile tables, especially for large codebases with many nested calls.

1.5 Viztracer

1.5.1 Usage in project

I ran VizTracer on my script using `viztracer -o result.html --open main.py`, which generated a `result.html` trace file. I then opened it to view the detailed execution timeline in the browser.



1.5.2 Insight from output

The output showed a timeline/flame-graph view where each function call was represented as a horizontal bar, with its width indicating duration and nesting showing the call hierarchy over actual time (not just aggregated totals). This made it easy to see the exact sequence and overlap of function calls, identify which calls happened concurrently or sequentially, and spot specific slow calls along the real execution timeline—something cProfile/Snakeviz’s aggregated view doesn’t show as clearly.

1.5.3 Maintenance use case

During maintenance, VizTracer can be used to trace a specific reported slow request or workflow end-to-end, capturing the exact sequence of operations and timing in that run—useful for diagnosing intermittent or timing-dependent issues that aggregate profilers might average out or miss.